You said:

QGroundControl stable version 5.1, A custom build to create own version of QGC , tell classes and interfaces for connection , telemetry, command, plan etc. and basic start guide

ChatGPT said:

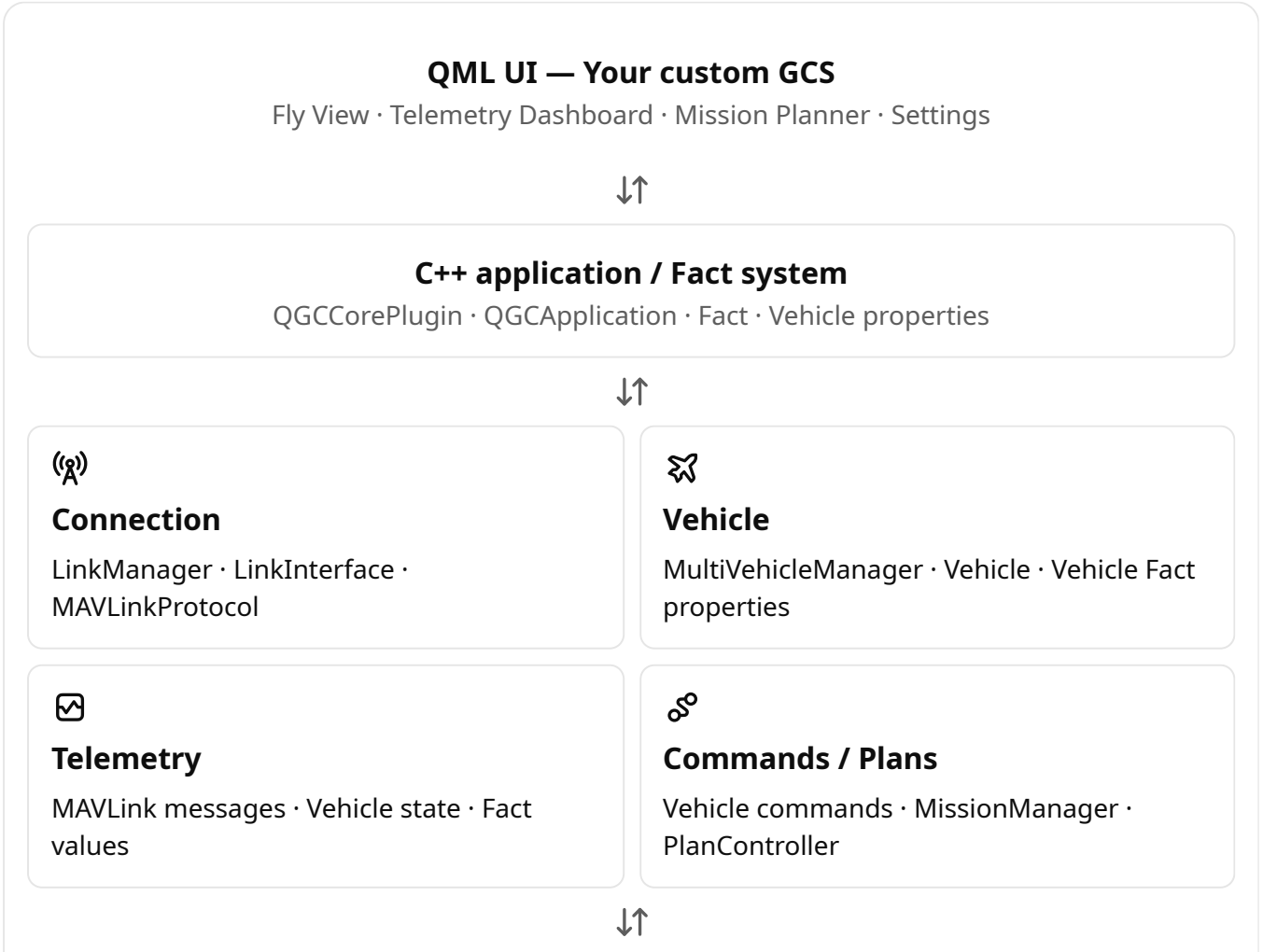
QGroundControl V5.1 — Custom GCS Development Guide

Your goal is to take the **stable QGroundControl V5.1 source code**, build your own customized Ground Control Station (GCS), and understand the C++ classes and interfaces responsible for vehicle connections, telemetry, commands, and mission planning.

QGC is built primarily with **C++ and Qt/QML**. You can reuse its existing MAVLink communication and vehicle-management architecture rather than implementing the entire GCS from scratch. The official repository supports custom builds and extensions.

GitHub +1

1. Understand the architecture



MAVLink transport

USB/serial · UDP · TCP · Telemetry radio

↓↑

Flight controller / simulator

PX4 or ArduPilot SITL / real autopilot

The key distinction is that **LinkManager handles communication links, Vehicle represents the connected aircraft, and higher-level managers handle its parameters, commands, and missions.** QML displays and interacts with those C++ objects; it should not independently implement a second MAVLink connection.

2. Important classes and interfaces

These are the main classes to study in the V5.1 source. Exact file locations can shift between releases, so use repository search to locate the declarations and implementations.

A. Connection and MAVLink communication

Class / interface	Responsibility
`LinkInterface`	Abstract interface for a communication link.
`LinkManager`	Manages communication links and their lifecycle.
`SerialLink` / UDP and TCP link implementations	Transport-specific communication.
`MAVLinkProtocol`	Processes MAVLink messages and protocol-level communication.
`MAVLink`	MAVLink message definitions and related protocol support.
`MultiVehicleManager`	Tracks discovered vehicles and the active vehicle.
`Vehicle`	Represents a connected UAV and exposes its state and operations.

Typical flow: USB or UDP link → MAVLink protocol → vehicle discovery → `Vehicle` object → UI.

Useful source directories to explore:

- `src/Comm` ↗
- `src/Vehicle` ↗
- `src/Utilities` ↗

B. Telemetry and parameters

Class / concept	What to learn
`Vehicle`	Vehicle position, attitude, armed state and other live state.
`Fact`	A typed value exposed to the UI, often with metadata and change notifications.
`FactGroup`	Groups related facts, such as GPS or battery values.
`ParameterManager`	Downloads, stores and updates autopilot parameters.
`MAVLinkProtocol`	Delivers incoming protocol messages to the relevant processing paths.

For example, your custom dashboard might display:

- Latitude and longitude
- Relative altitude
- Roll, pitch and yaw
- Battery voltage and remaining percentage
- GPS fix and satellite count
- Armed state and flight mode

The preferred design is to **consume the existing `Vehicle` properties and Fact objects** rather than writing a separate UDP receiver for each telemetry field.

C. Commands and flight control

Class / concept	Responsibility
`Vehicle`	High-level vehicle operations and command dispatch.
`MAVLinkProtocol`	Protocol processing for command-related messages and replies.
`FirmwarePlugin`	Firmware-specific behavior and command handling.
MAVLink command definitions	Define command IDs, parameters and acknowledgements.

`ParameterManager`	Handles parameter operations, which are different from ordinary flight commands.

Examples of MAVLink operations include requesting a mode change, arming or disarming, requesting telemetry, and sending a command with acknowledgement handling.

Important: A command is not the same thing as telemetry. A command requests an action; an acknowledgement reports whether it was accepted or rejected. The vehicle's resulting state should be confirmed from telemetry.

D. Mission planning

Class / concept	Responsibility
`PlanMasterController`	Coordinates planning components and plan state.
`MissionController`	Manages mission items and their planning representation.
`MissionManager`	Handles mission transfer and related vehicle-side interactions.
`VisualMissionItem`	Base abstraction for visual mission items.
`SimpleMissionItem`	Represents a simple MAVLink mission command.
`ComplexMissionItem`	Supports higher-level items such as survey-style plans.
`PlanView` and related QML	Presents mission planning controls and map interaction.

The mission workflow is generally:

1. Create mission items in the planning UI.
2. Validate the plan and its parameters.
3. Upload the mission to the vehicle.
4. Monitor transfer results.
5. Start or manage execution using the appropriate vehicle controls.
6. Observe mission progress from incoming telemetry.

Mission upload and mission execution are separate operations. Uploading a plan does not itself mean the aircraft has started flying it.

3. Exact source files to study first

These are useful entry points in the V5.1.5 source tree. The official stable release is on the `v5.1.5` tag ↗. GitHub +1

`src/Comm/`

Start with `LinkInterface`, `LinkManager`, and the transport implementations. Learn how QGC opens and manages links.

Browse communication source ↗

`src/Vehicle/Vehicle.h` and `Vehicle.cc`

The main vehicle abstraction. Study its QML properties, signals, command methods and manager objects.

Open Vehicle.h ↗ · Open Vehicle.cc ↗

`src/FactSystem/`

Learn `Fact`, `FactGroup` and `ParameterManager` to understand how telemetry and parameters reach the UI.

Browse FactSystem ↗

`src/MissionManager/`

Explore `MissionManager`, `PlanManager`, `MissionController` and `PlanMasterController`.

Browse mission source ↗

`QGCCorePlugin` and custom-build configuration

Learn how to add your own branding, customize navigation and override supported UI or firmware behavior.

Official custom-build guide ↗

Note: these are source-navigation starting points. Verify each filename against the stable tag, rather than assuming every class is in the same directory or that every transport has a standalone class with the obvious name.

4. Build your own QGC V5.1.5

Since you previously wanted to use Windows 11 with WSL for development, there are two reasonable options:

- **Native Windows build:** best if you want to run and debug the Windows application directly.
- **WSL/Linux build:** useful for learning and Linux development, but running the GUI requires a working graphical environment.

For a Windows 11 custom desktop GCS, I recommend starting with the native Windows build.

Step 1 — Clone the stable source

Open PowerShell or a suitable developer terminal:

```
git clone --branch v5.1.5 --recurse-submodules `
  https://github.com/mavlink/qgroundcontrol.git

cd qgroundcontrol
git submodule update --init --recursive
```

Use the Git repository, not just the downloaded source ZIP, because submodules are needed for the build. GitHub +1

Step 2 — Install the required development tools

Install:

- Qt version specified in the V5.1.5 build configuration
- Qt Creator, or use CMake from the command line
- Visual Studio 2022 with the C++ desktop development tools
- Git and Python
- Ninja, if using the Ninja build generator
- The dependencies required by the QGC setup scripts

For this stable tag, consult the V5.1.5 build guide ↗ for the exact Qt version and setup requirements. Do not substitute the latest Qt version without checking compatibility.
GitHub

Step 3 — Configure and build

From the QGC source directory, use the Qt CMake tool matching your installed Windows kit. For example, from a Visual Studio 2022 x64 developer terminal:

```
python tools/setup/install_dependencies --platform windows

git submodule update --init --recursive

& "C:\Qt\<QT_VERSION>\msvc2022_64\bin\qt-cmake.bat" `
  -S . -B build -G Ninja `
  -DCMAKE_BUILD_TYPE=Debug

cmake --build build --parallel 8
```

Replace `<QT_VERSION>` with the exact version required by the tag. If the build configuration uses another generator or toolchain, follow the stable guide.

Step 4 — Run your build

The executable location depends on the configured generator and build type. With the example configuration, check:

```
Get-ChildItem build -Recurse -Filter QGroundControl.exe
```

Run the resulting executable. Confirm that the stock application starts and that a simulated vehicle can connect before making custom changes.

5. Customize QGC without rewriting its core

For your first custom version, keep the existing communication, telemetry and mission systems intact.

1. **Branding:** change the application name, icon, splash screen and appropriate resources.
2. **Simplify the UI:** use custom-build plugin overrides to hide irrelevant pages or limit supported firmware options.
3. **Add a telemetry dashboard:** create a QML page that consumes the active vehicle and its Fact properties.
4. **Add your own controls:** expose suitable C++ methods to QML through the established QGC architecture, with safety checks and command feedback.
5. **Extend mission planning:** reuse the existing planning controller and mission item model instead of inventing another mission protocol.

The official custom-build mechanism specifically supports plugin overrides, custom QtQuick interfaces, toolbar changes and custom Fly View overlays. GitHub

6. Recommended learning order

1 **Connection and vehicle discovery**

Learn `LinkInterface` → `LinkManager` → `MAVLinkProtocol` → `MultiVehicleManager` → `Vehicle`.

Milestone: connect QGC to SITL and identify the active vehicle.

2 **Telemetry**

Learn `Vehicle`, `FactGroup`, `Fact` and the relevant GPS and battery groups.

Milestone: show live position, altitude, attitude and battery status.

3 **Commands**

Trace a supported command from a QML action through `Vehicle` to MAVLink and its acknowledgement.

Milestone: understand command acceptance and state confirmation using SITL.

4 **Mission planning**

Trace `PlanMasterController`, `MissionController`, `MissionManager` and `PlanManager`.

Milestone: create a mission in the UI, upload it to SITL and observe mission progress.

5 **Custom GCS**

Add your own page and branding using the custom-build plugin system.

Milestone: your application launches with a custom dashboard and the original QGC communication stack.

Use PX4 SITL ↗ or ArduPilot SITL ↗ while learning. Validate commands and mission changes in simulation before testing with real aircraft.

My recommendation: start with `Vehicle.h`, then trace how `MultiVehicleManager` creates and exposes the active vehicle, and finally inspect the QML Fly View. That path connects the C++ backend to the interface you will customize.